Authentication for the workers-hono-svelte profile — research report

Nov 5, 2026 · @Rob

Summary

Researched 2026-11-05. Recommendation: **the api authenticates (option a)**. The gate lives in the Hono Worker, reads an opaque session id from an HttpOnly cookie, looks it up in a **D1 `sessions` table** joined to `users`, and attaches the actor or refuses in the error envelope. The BFF forwards the browser's cookie over the service binding and decides nothing. Sessions are **hand-rolled** (the Lucia guide pattern) with `arctic` for OAuth, `@simplewebauthn/server` for passkeys and `@oslojs/*` for encoding and tokens. No framework-owned auth library sits in the request path.

Why this and not the alternatives, in one line each:

- **BFF authenticates (b):** moves the authority out of the api, so a CLI or a second client needs a second authority, and the api must trust a header. That is the one-authority rule broken at the first step.
- **Both (c):** two deciders by construction; the only defensible form of (c) is (a) plus a BFF *cache* that is never consulted for a decision.
- **Separate auth Worker (d):** earns its keep only when more than one api with different owners must share identities. For one api it adds a hop per request or a token format, a second OpenAPI document and a second local dev process, and gives nothing the api's `auth` area cannot.
- **KV sessions:** eventual consistency (up to ~60 s between locations) makes sign-out and disable non-immediate. Fine for a cache, wrong for the authority.
- **Durable Object per user as session authority:** correct semantics but a cross-region hop per request, a second storage system, and the owner's scoped-query ownership model does not apply. Worth it only for live revocation fan-out at large scale.
- **Stateless signed cookie:** no immediate revocation and no "sign out everywhere"; disable becomes advisory until expiry.

Versions as of 2026-11-05 (npm registry): Svelte 5.57.3, **SvelteKit 3.1.1** (3.0 shipped 2026-10-02; the brief says 2.x, see §3), `@sveltejs/adapter-cloudflare` 8.0.2, Hono 4.13.16, Wrangler 4.158.0, `@cloudflare/workers-types` 5.20261104.1, better-auth 1.7.18,

`@auth/sveltekit` 1.11.3 / `@auth/core` 0.41.3, arctic 3.7.0, `@oslojs/crypto` 1.0.1, `@simplewebauthn/server` 14.0.3, `@openauthjs/openauth` 0.4.3 (last published 2025-03-22), lucia 3.2.2 (deprecated).

The three decisions the owner must still make

1. **Host topology for the cookie.** Either the BFF and api share one hostname and are split by route (`app.example.com/*` → BFF, `app.example.com/api/*` → api), or they sit on sibling subdomains with the cookie scoped to the parent domain. Same-host is simpler and gives a `Host`-prefixed cookie; sibling subdomains make every browser→api call cross-origin and force a CORS and CSRF story. The report assumes same-host route split (§1, §5). This is a product decision because it fixes what the public URL of the api is.
2. **Password hashing policy.** Workers caps WebCrypto PBKDF2 at 100,000 iterations (not configurable; verified by multiple independent reports, not by a Cloudflare doc I could find — see §3). OWASP's floor for PBKDF2-HMAC-SHA256 is 600,000. The choices are: ship passwords on PBKDF2-100k and accept being below guidance; run argon2id via WASM inside the Worker's CPU budget on the Paid plan; or launch without passwords (passkeys + Google + magic link) and add passwords later. The report recommends the third, with argon2id-via-WASM as the fallback if passwords are a hard requirement.
3. **Admin access: Cloudflare Access or the app's own role.** Access can front a path such as `/admin/*` without touching the rest of the Worker and gives SSO plus device posture for free (up to 50 users). But it is a second decider for *who may reach admin*, and the api must still verify the `Cf-Access-Jwt-Assertion` and map it to an actor. The report recommends Access as defence-in-depth on the admin surface *and* the app's `role` as the one authority an operation consults; §4 and §6 lay out the alternative.

What the recommended architecture looks like

Concern	Where	Mechanism
Authentication	api gate (Hono Worker)	Read `sid` cookie → `SELECT … FROM sessions JOIN users WHERE id = ? AND expires_at > ?` → attach actor or refuse
Authorization	operation layer	Spec table per operation; store queries scoped by actor; wrong owner reads as not found

Concern	Where	Mechanism
Session store	D1 `sessions`	Opaque 256-bit id, hashed before storage, sliding expiry, `user_id`, `created_at`, `last_seen_at`, `user_agent`
Credentials	D1 `auth` area	`passwords`, `passkeys`, `oauth_accounts`, `verification_tokens` tables
Cookie	api sets it	`__Host-sid`, HttpOnly, Secure, SameSite=Lax, Path=/
BFF → api	service binding `fetch`	Forwards `Cookie` header verbatim; forwards `Set-Cookie` back; never reads the session itself
CLI (consumer)	bearer token	Separate `api_tokens` table; same gate, second credential source
CLI (provider)	in-process	Gate runs against the same `auth` store; actor is the local operator
Sign-up / sign-in abuse	gate + platform	Turnstile on the forms; Rate Limiting binding keyed by account and by IP before any hash is computed
Email	platform	Cloudflare Email Sending (public beta since 2026-04-16) with a third-party fallback
Admin	platform + operation	Access on `/admin/*`; `role = 'admin'` checked by operations

Round trips per entry chain under this design: page load = 1 BFF→api call carrying the cookie, 1 D1 read for the session, then the operation's own reads; api call from the browser = 1 D1 read for the session plus the operation. The budget test asserts "gate costs exactly one store read" and "sign-out makes the next request refuse".

1. Where does authentication run

The api is the only component that can be the authority without breaking the one-authority rule, because it is the only component every consumer (browser via BFF, CLI, future mobile client) already passes through. Everything else in this section follows from that.

	(a) api authenticates	(b) BFF authenticates	(c) both, shared verification	(d) separate auth Worker
Authority	api gate	BFF `hooks.server.ts`; api trusts an assertion	Two; whichever ran last	Auth Worker; api and BFF are relying parties
How the actor reaches an operation	Gate attaches it to the request context; entry point passes it in	BFF puts `X-User-Id` (or a signed JWT) on the service-binding call; api gate decodes it	Same as (a) plus a BFF copy for rendering	api gate calls auth Worker (RPC or fetch) or verifies a token it issued
Cookie origin, same host route-split	api sets `__Host-sid`; BFF forwards it	BFF sets it; api never sees a cookie	Both read it	Auth Worker sets it on the shared host
Cookie origin, sibling subdomains	`Domain=example.com`, `SameSite=Lax`; browser→api calls are cross-origin and need CORS + CSRF token	BFF-only cookie, `__Host-` prefix possible	As (a)	`Domain=example.com`, set by `auth.example.com` after redirect
Different zones	Not shareable; the BFF must proxy every api call	Works, because only the BFF has the cookie	As (b)	Redirect-based (OIDC-style) only
Provider CLI	Runs the same gate in-process against the same store	Must reimplement the gate; the api's trust-a-header path is unusable	Same as (a)	Needs the auth Worker reachable or a local stub
Consumer CLI	Bearer token, same gate	Must call the BFF or get a second trust path into the api	Same as (a)	OAuth device flow against the auth Worker
Round trips, page load	1 BFF→api + 1 D1 read in the gate	1 D1 read in the BFF + 1 BFF→api	2 D1 reads unless cached	1 BFF→auth + 1 BFF→api (+ 1 api→auth unless token-based)
Round trips, direct api call	1 D1 read	Not possible without the BFF	1 D1 read	1 api→auth or 0 with a verified token
Auth store down	Gate refuses everything; the error envelope is the only output	BFF 500s; api would accept a forged header if it leaked	Both fail	Token-based: existing tokens still verify; lookup-based: everything fails

	(a) api authenticates	(b) BFF authenticates	(c) both, shared verification	(d) separate auth Worker
OpenAPI	Auth endpoints and the security scheme live in the one document	Sign-in is outside the api; the document cannot describe how to get a credential	Two partial documents	A second document for the auth Worker
Local development	`wrangler dev` with two Workers, already the case	Same	Same	A third process, a third config, a third set of secrets

(a) in detail

The gate in the Hono Worker runs before routing. It reads the `sid` cookie (or an `Authorization: Bearer` header for CLI tokens), does one indexed D1 read, and returns either `{ actor }` or an error envelope. It never throws. Authorization stays per operation. The BFF's `handleFetch` (SvelteKit) or a thin wrapper around `env.API.fetch()` copies the inbound `Cookie` header onto the outbound request and copies any `Set-Cookie` back. Hono's `getCookie`/`setCookie` helpers are enough; no middleware package is needed.

The cookie question is the only awkward part. A service binding call has no origin, so cookies work by header copying, not by browser rules. What the *browser* needs is that the cookie it holds for the BFF's host is also the cookie the api expects. That is trivially true if the api is reachable on the same host (`/api/*` routed to the api Worker, the rest to the BFF), which Cloudflare routes support directly. With sibling subdomains the cookie must carry `Domain=example.com` and loses the `__Host-` prefix, and any browser-direct call to the api is cross-site, which brings CORS pre-flights and the CSRF story in §5. There is no reason for the browser to call the api directly at all if the BFF proxies, in which case the cookie can be `__Host-` scoped to the BFF host and still forwarded over the binding. **That is the simplest correct design: the cookie is the BFF host's, the api only ever sees it over the binding.** Note that if the api is *also* public (for the CLI), it then has two credential types: cookie over binding, bearer from the network.

(b) in detail

SvelteKit's `hooks.server.ts` is where every SvelteKit auth library wants to live. It fits the framework well. It fits this design badly for three reasons. The actor's source of truth is now a Worker that renders pages; a provider CLI that runs operations in-process has no gate at all. The api must accept an identity assertion it did not mint, which means either trusting the service binding unconditionally (fine today, a footgun the day a second caller is bound) or verifying a signature (now the BFF is a token issuer and the api a verifier — you have built (d) without the separation). And the OpenAPI document can no longer describe how to obtain a credential, because sign-in is not an api endpoint.

(c) in detail

Two components that both decide is the definition of a defect under the design rules. The only version worth keeping is (a) plus a BFF-side *memo*: the BFF may remember "this request was accepted by the api as user U" for the duration of one render to avoid a second call. It never consults that memo to refuse. If the BFF needs the actor before it calls the api (to pick a layout, say), it calls the api's `GET /me` endpoint, which is just another operation behind the gate.

(d) in depth

A dedicated auth Worker is the OpenAuth model (SST) and the Cloudflare Access model: an issuer that authenticates once and hands out tokens; every other service is a relying party that verifies. On Cloudflare specifically it earns its separation when:

- more than one api, owned by different teams or repos, must accept the same identity;
- third parties must sign in to your users ("login with X" — you are an OAuth provider);
- the identity surface changes on a different cadence from the product (a platform team ships passkeys; product teams never touch it);
- you need an issuer URL with `.well-known` metadata, device-code flows for CLIs, and token introspection.

It costs:

- **One-authority rule.** The auth Worker decides "who is this"; the api decides "is this session still good" only if it checks back (a hop) or accepts the token's expiry as truth (no immediate revocation unless tokens are short-lived with refresh, which pushes complexity to every client). There are now two places where "this user is disabled" must be known.

- **OpenAPI.** The api document declares a `bearer` security scheme and points at an external issuer. The issuer has its own document (or none; OpenAuth exposes OAuth metadata, not OpenAPI). The spec-address-and-check rule now spans two repos or two packages.
- **Local development.** A third `wrangler dev` process, its own D1 or KV, its own secrets, and a redirect flow that must work on `localhost` with two ports. Miniflare's multi-worker dev handles bindings, but OAuth redirects to a second local origin are a recurring source of friction.
- **Latency.** Token verification is local (JWKS cached), so steady-state cost is near zero; but every sign-in is now two redirects and the first request after a cold start fetches JWKS.

For a monorepo with one api and one BFF, (d) is premature. The cheapest hedge is to keep the `auth` area's *interface* clean enough that it could be lifted into its own Worker later: no operation imports from `auth` except through the gate's actor type and a small `AuthStore` port. If that discipline holds, moving to (d) is a deployment change, not a rewrite.

What the budget test asserts under (a)

- Gate: exactly 1 store read per authenticated entry; 0 when the cookie is absent (refuse without touching the store).
- Page load: 1 BFF→api call for the data the page needs; the gate's read is inside it.
- Sign-in: 1 credential-store read + 1 session write + 1 user read (can be one statement with a join).
- Sign-out this device: 1 delete. Sign-out all: 1 delete by `user_id`.

2. Session storage on Cloudflare

D1 is the right store for this system at both scales: it is already the database, it is strongly consistent at the primary, revocation is a row delete, and the ownership-by-scoped-query rule applies to sessions exactly as to everything else. KV is disqualified as the authority by its consistency model. A Durable Object becomes interesting only when per-user fan-out (live revocation to open connections, device lists with presence) is a product feature.

	D1 table	Workers KV	Durable Object (per user)	Durable Object (one coordinator)	Signed stateless cookie
Status (2026-11-05)	GA since 2024-04; read replication **public beta** since 2025-04-10 (Sessions API)	GA; re-architected 2025-08 (hybrid storage, <5 ms p99 hot reads)	SQLite backend GA 2025-04-07; on Free plan 2025-04-07; storage billing started 2026-01-07	Same	n/a
Consistency	Strong at primary; replicas are async and need the Sessions API with a bookmark to be sequentially consistent	**Eventual.** Writes visible immediately at the writing location; up to ~60 s elsewhere (default `cacheTtl` 60 s); negative lookups are cached too	Strong; single-threaded per object	Strong; single point of serialization	n/a (nothing to be consistent with)
Sign-out / disable	Immediate: delete the row; next gate read misses	**Not immediate** from other locations for up to the cache TTL; a disabled user keeps working for up to a minute elsewhere	Immediate	Immediate	**Impossible** before expiry without a deny-list, which is a store again
Read latency from a Worker	Primary: one region, so 5–80 ms depending on distance; replica (beta): single-digit to ~20 ms nearby	Hot: ~1–5 ms; cold: tens of ms	~1 ms if the object is co-located; one inter-region hop (30–150 ms) if not. Object lives where first created unless a location hint is given	Same, but every session read in the world goes to one place	0 (HMAC verify is microseconds)
Cost, a few hundred users	Free tier (5M rows read/day)	Free tier (100k reads/day)	Free tier	Free tier	0

	D1 table	Workers KV	Durable Object (per user)	Durable Object (one coordinator)	Signed stateless cookie
Cost, a few hundred thousand users (say 20M gate reads/month)	Included in Paid: 25B rows read/month; writes 50M/month included then $1/M	10M reads included then $0.50/M → ~$5; writes $5/M	1M requests included then $0.15/M → ~$3 plus duration (GB-s) and SQLite rows (same rates as D1)	Same but one object = throughput ceiling and a hotspot	0
TTL	Your `expires_at` column + a cron or lazy delete	Native `expirationTtl` (min 60 s)	Alarms API; or `expires_at` in SQLite	Same	`exp` claim
Revocation of *all* devices	`DELETE WHERE user_id = ?`	List by prefix (eventually consistent) then delete each	Clear the object's table	One delete	Bump a per-user `version` in a store → a store again
Budget test asserts	Gate = 1 read; sign-out visible on next request in the same test	Gate = 1 read; **cannot** assert sign-out visibility across locations	Gate = 1 RPC; sign-out visible next request	Same	Gate = 0 store reads; **cannot** assert revocation

KV

The 2025 re-architecture (hybrid storage with R2, 40× throughput, <5 ms p99 for hot keys) changed latency, not semantics. Cloudflare's own docs still say reads are eventually consistent, that changes take up to 60 s to be visible in other locations, and that strong consistency needs Durable Objects. For a session *cache* in front of D1 it is fine; for the authority it means a disabled account or a stolen session stays live for a minute from any location other than the one that revoked it. The owner's rule that a wrong owner reads as not found cannot be stated for KV because there is no scoped query, only a key.

Durable Objects

SQLite-backed DOs are the recommended class type; KV-backed remain for compatibility. Each DO is single-threaded with input/output gates, so a per-user object serializes all of that user's session operations, which is exactly right for "sign out

everywhere" and for counting failed logins. Placement is the problem: a DO is created near its first caller and stays there (location hints only steer initial placement, and until 2025 there was no migration). A user who signs up in Los Angeles and travels to Berlin pays a transatlantic hop on every gate read. Hibernation makes idle objects free, but a wake-up costs cold-start time. **A DO per user as the session authority is not an anti-pattern, it is a different product**: it buys live revocation fan-out and per-user serialization at the cost of a second storage system and losing D1's cross-user queries (admin "list active sessions for everyone" becomes a scatter). One coordinator DO for all sessions is an anti-pattern: a global serialization point for every authenticated request. Cloudflare's own best-practice guidance (changelog 2025-12-15, "rules of Durable Objects") says to shard by natural key and never funnel all traffic through one object. Alarms on SQLite-backed DOs were disabled briefly in 2024-09 and restored 2024-09-27; nothing current.

The reasonable DO use *here* is not sessions but **sign-in throttling**: a per-account DO (or the Rate Limiting binding, §4) that counts failures with strong consistency, so a credential-stuffing run cannot exploit D1's lack of atomic increment-and-check across regions.

D1 specifics

- D1 has one primary region per database. With the BFF and api in the same account the Worker runs at the edge and the D1 call crosses to the primary; this is the latency floor for the gate and is the same floor every operation already pays. Read replication (beta) lowers it for reads but needs a bookmark passed via cookie or header to avoid reading your own sign-in from a lagging replica; using it for the session read alone is a reasonable later optimization, not a day-one requirement.
- Store the session id **hashed** (SHA-256 of the 256-bit random id) so a database dump does not yield live cookies. Index on the hash. This is the Lucia guide's current pattern and what the Copenhagen/Auth Book recommends.
- Sliding expiry: extend `expires_at` when more than half the window has passed, so most gate reads are read-only. Rows written are 50× more expensive than rows read on D1.
- Expired-row cleanup: a Cron Trigger deleting `WHERE expires_at < now` daily; cost is negligible at either scale.

Stateless signed cookie

JWT or HMAC-signed value, no store. Verify costs nothing and survives any outage. It fails the design on revocation: `disabled` on the user cannot take effect until expiry, and "sign out all devices" needs either a per-user version in a store (then you have a store) or short lifetimes with refresh tokens (then you have a store for the refresh tokens and a second credential to protect). It is the right choice for the CLI's short-lived tokens *derived from* a stored credential, not for browser sessions.

Status changes in the last eighteen months (2025-05 → 2026-11)

Date	Product	Change
2026-07-07	Workflows	Billing updates announced (steps and storage billing start date)
2026-04-16	Email Service	Email Sending public beta (private beta 2025-09-25)
2026-02-04	Queues	Available on the Free plan
2026-01-07	Durable Objects	SQLite storage billing enabled
2025-12-15	Durable Objects	Best-practices guide published
2025-10-16	Durable Objects	Data Studio (beta) for SQLite-backed objects
2025-09-19	Workers	Rate Limiting binding GA (`ratelimits` config; `unsafe.bindings` deprecated)
2025-08	KV	Re-architecture: hybrid storage, <5 ms p99 hot reads, read-your-own-write within a location
2025-08-21	Durable Objects	`getByName()` API

Earlier but relevant: D1 read replication beta 2025-04-10; SQLite-in-DO GA and DOs on the Free plan 2025-04-07; Workflows GA 2025-04; Queues GA 2024-10 (from memory, not re-verified).

3. Identity methods and their libraries

None of the framework-level libraries fits a design where the api owns the decision, because each one owns its routes, its cookie and its schema as a package. The primitives (arctic, oslo, simplewebauthn) fit, because they decide nothing. The lab should compose primitives behind the `auth` area and treat the Lucia guide as the reference implementation of sessions.

What each method needs from the platform

Method	Secrets	Redirect / origin binding	Email delivery	Store tables	Notes
Email + password	Pepper (optional)	None	Verification and reset mail	`passwords(user_id, hash, params)`, `verification_tokens`	Hashing cost is the design constraint (below)
Passkeys / WebAuthn	None	**Relying-party id = the host** (`example.com` covers subdomains; `localhost` in dev); origin must match exactly	None for sign-in; recovery still needs mail or a second factor	`passkeys(id, user_id, public_key, counter, transports, backed_up)`	Challenge must be single-use and stored server-side (D1 row or short-lived signed value)
OAuth / OIDC (Google first)	Client id + secret per provider	Exact redirect URI per environment, registered in the provider console	None	`oauth_accounts(provider, provider_user_id, user_id)`	PKCE + `state` mandatory; Google requires HTTPS redirect except localhost
Magic link	None	Link target must be your host; token in path or query	**Required**, and is the whole security of the method	`verification_tokens(hash, user_id, purpose, expires_at, used_at)`	Token entropy ≥ 128 bits, hashed at rest, single use, 10–15 min expiry

Method	Secrets	Redirect / origin binding	Email delivery	Store tables	Notes
Admin via Cloudflare Access	Access AUD tag; JWKS public	Access app on a hostname or path; Access sets the cookie	None	None in the app; map Access identity → `users.role`	api must verify `Cf-Access-Jwt-Assertion` against the team's JWKS; never trust the header's presence alone

Library state on Workers (versions as of 2026-11-05)

Library	Version	Workers	Owns routes?	Owns cookie?	Owns schema?	Fits "api decides"?
Better Auth	1.7.18 (Vercel acquired Better Auth 2026-07-07; MIT, continues)	Yes; D1 via Drizzle/Kysely adapters; has Cloudflare-specific fixes (AsyncLocalStorage cold-start race fixed 1.6.24)	**Yes.** A single `auth.handler(request)` mounted at `/api/auth/*`; endpoints are the library's	**Yes.** Sets and names the session cookie; `__Secure-` prefix configurable	**Yes.** `user`, `session`, `account`, `verification` tables with its column names; `auth generate` emits migrations	Partially: it can be mounted *inside* the Hono api, which keeps the authority in the api Worker, but the gate must call `auth.api.getSession()` and the sessions table is Better Auth's, not yours. Authorization plugins (organization, admin) are a second decider and must not be used
Auth.js / `@auth/sveltekit`	1.11.3 on `@auth/core` 0.41.3	Yes	Yes, in `hooks.server.ts` (`/auth/*`)	Yes	Yes, via adapter	**No.** Lives in the BFF by construction (option b). Project is in maintenance under Better Auth since 2025-09-22; maintainers recommend Better Auth for new work; July 2026 advisories fixed in 0.41.3

Library	Version	Workers	Owns routes?	Owns cookie?	Owns schema?	Fits "api decides"?
Lucia	3.2.2 (deprecated 2025-03)	n/a	—	—	—	**As a guide, yes.** lucia-auth.com is now a learning resource updated 2026-07 with a single-file `auth_session.ts` reference; it prescribes nothing about routes or framework. This is the recommended pattern
Oslo (`@oslojs/crypto`, `/encoding`, `/webauthn`, `/oauth2`, `/jwt`)	crypto 1.0.1 (2026-07), encoding 1.1.0	Yes, pure JS, no Node deps	No	No	No	**Yes**
Arctic	3.7.0 (2026-07-29)	Yes	No; you write the redirect and callback handlers	No	No	**Yes.** 50+ providers; `generateState`, `generateCodeVerifier`, `validateAuthorizationCode`
`@simplewebauthn/server`	14.0.3 (2026-09-25)	Yes since v9 removed Node-only crypto; v13+ ESM-only	No	No	No; you store the credential	**Yes**
OpenAuth (`@openauthjs/openauth`)	0.4.3, **last published 2025-03-22**; repo moved sst → toolbeam → anomalyco	Yes, runs on Workers with KV storage	Yes; it *is* an issuer (option d)	Yes	KV-shaped store, not relational	**No** for this topology; the right tool only if (d) is chosen, and its release cadence (none in 19 months) is a risk

Two observations on Better Auth, since it is the obvious default in 2026: it is the best-maintained option and its Cloudflare support is real, but adopting it means the `auth` area's schema address is a generated migration you do not author, the sessions table is not the one the owner designed, and the spec-and-check rule has nothing to check against. It also wants to answer authorization questions through plugins, which the rules forbid. If the owner wants a library anyway, mount it inside the Hono api (never in SvelteKit), use only its core session + credential endpoints, and write the gate as a thin call to `auth.api.getSession({ headers })`. That keeps the authority in the api at the cost of schema ownership.

SvelteKit 3 note

The brief names SvelteKit 2.x. SvelteKit 3.0 shipped 2026-10-02 (Vite 8/Rolldown required, config moves into `vite.config.ts`, `$lib` → `#lib`, all errors routed through `handleError`). `@sveltejs/adapter-cloudflare` 8.0.2 is the matching adapter. Nothing in

the auth design depends on the major version: `hooks.server.ts`, `handleFetch`, `event.cookies` and `platform.env` are unchanged. The owner should decide whether the lab tracks 2.x or 3.x before the auth work starts, because the migration touches every import.

Password hashing on Workers

- **CPU budget.** Free plan: 10 ms CPU per request, which rules out any serious password hash. Paid: 30 s default, raisable to 5 min via `limits.cpu_ms`. Hashing is CPU time, not wall time, so it is billed ($0.02 per million CPU-ms after 30M included).
- **WebCrypto PBKDF2.** Supported. Multiple independent reports (2024–2026) state the runtime rejects more than **100,000 iterations**; I could not find this limit in Cloudflare's own docs, so treat it as verified-by-practice, not documented. OWASP's Password Storage Cheat Sheet currently says PBKDF2-HMAC-SHA256 at **600,000** iterations (210,000 for SHA-512) and prefers argon2id (m=19 MiB, t=2, p=1) or scrypt (N=2^17, r=8, p=1) or bcrypt (work factor 10). PBKDF2-100k is therefore below guidance by 6×.
- **argon2 / bcrypt via WASM.** `@noble/hashes` 2.4.0 ships pure-JS argon2id and scrypt (slow but portable); `hash-wasm` ships argon2id compiled to WASM and runs on Workers; `@node-rs/argon2` does **not** (native addon). At OWASP's minimum argon2id parameters a Worker spends roughly 50–150 ms CPU per hash in WASM (community benchmarks; not re-measured for this report), well inside a Paid budget and acceptable for a sign-in path that is already rate-limited. The 128 MB isolate memory cap is not a problem at 19 MiB. Note `scrypt` is *not* in WebCrypto on Workers.
- **Recommendation.** Launch without passwords if the product allows it (passkeys + Google + magic link covers sign-in and recovery). If passwords are required, argon2id via WASM on the Paid plan, with a `params` column so the work factor can be raised and hashes re-wrapped on next sign-in. Do not ship PBKDF2-100k as the permanent answer; if used as a bridge, record the iteration count per row.
- **Where it runs.** The hash is computed by the `auth` operation on the api, never on the BFF, and only after the Rate Limiting check has passed (§5), so an attacker cannot buy your CPU time with a dictionary.

4. The Cloudflare product map for the whole flow

Six products carry the design: Workers, D1, the Rate Limiting binding, Turnstile, Email Sending and Access. The rest are either a later optimization (KV cache, DO throttle, Queues for mail) or not needed (Workflows, Secrets Store).

Product	Role in the flow	Should it?	Status / plan notes
Workers	Host the gate, the `auth` operations, the BFF	Yes	Paid plan required in practice: 10 ms CPU on Free rules out hashing; `limits.cpu_ms` up to 300,000
D1	Users, credentials, sessions, verification tokens	Yes, the authority	One primary region; read replication beta; 5 GB included on Paid
KV	Optional read-through cache of `session → actor` with a short `cacheTtl`	Only at the larger scale, and only as a cache the gate may miss, never as the record of truth	Eventual consistency (§2); if used, sign-out must also delete the KV key and accept up to 60 s of staleness elsewhere
Durable Objects	Per-account failed-login counter; later, per-user live revocation	Throttle: maybe, if the Rate Limiting binding's per-location semantics are not enough. Sessions: no	SQLite-backed, GA; Free plan OK for the counter
Queues	Decouple "send verification mail" from the request	Yes once mail volume or provider latency matters; not day one	On Free plan since 2026-02-04; 1M ops/month included on Paid; a send is ~3 ops

Product	Role in the flow	Should it?	Status / plan notes
Workflows	Multi-step flows with waits (e.g. "remind after 24 h if unverified")	No for auth; the flows here are one step	GA 2025-04; step billing changes announced 2026-07-07
Email Routing / Email Workers (receiving)	Inbound mail to a Worker	No role in auth	GA; free
Email Sending (Email Service)	Verification, magic link, reset mail from `env.EMAIL.send()`	Yes, with a fallback sender	**Public beta since 2026-04-16** (private beta 2025-09-25). Needs your domain on Cloudflare with DKIM/DMARC records it provisions. Sending to arbitrary recipients requires Workers Paid; before onboarding it sends only to verified addresses. Binding address shape is `{email, name}`; REST shape is `{address, name}`. `wrangler dev` simulates sends unless `remote: true`. A third-party sender is **not** unavoidable any more, but a beta product with no public deliverability history should not be the only path for a security-critical mail; keep a Resend/Postmark/SES adapter behind the same `Mailer` port
Turnstile	Bot check on sign-up, sign-in, magic-link request, password reset	Yes	Free for unlimited widgets on all plans (Enterprise tier adds analytics and SLA); server-side `siteverify` call from the api operation, not from the BFF; the token is single-use and expires in 300 s
Rate Limiting binding	Throttle by `email` and by `cf-connecting-ip` before hashing or sending mail	Yes, first line	**GA 2025-09-19**; config key `ratelimits`, `period` must be 10 or 60 s; **limits are per Cloudflare location, not global**, so a distributed attacker gets N× the limit; good enough to protect CPU, not enough alone for lockout policy (pair with a D1 or DO counter per account)
WAF rate limiting rules	Path-level throttle on `/api/auth/*` at the zone	Yes, as the outer layer	Available on Free (1 rule) and Pro+; counts by IP, JA3/JA4, headers; applies before the Worker runs so it also protects against CPU-time billing abuse

Product	Role in the flow	Should it?	Status / plan notes
Cloudflare Access	Admin gate on `/admin/*` (BFF) and `/api/admin/*` (api)	Yes as defence in depth; not as the only authority	Zero Trust Free plan covers 50 users. Can front a **specific hostname or path** of a Worker via a self-hosted application (path wildcards supported; port numbers and query strings not). Workers dashboard now has "Protect this Worker" and account-wide "Protect all Workers" (2026). Access adds `Cf-Access-Jwt-Assertion`; the api must verify it against `https://<team>.cloudflareaccess.com/cdn-cgi/access/certs` with the app's AUD. For api→api or CLI→admin, Access **service tokens** work without a browser
Secrets Store	Account-level secrets shared by both Workers (OAuth client secrets, pepper)	Optional	Still beta at last check (2026-04); binding is `secrets_store_secrets` with async `.get()`; deploy token needs *Account Secrets Store Edit*. Per-Worker `wrangler secret put` is GA and sufficient for two Workers
Service bindings (`fetch`)	BFF → api carrying the cookie	Yes, today's choice	Zero-cost, same-thread when co-located; the api sees a normal `Request`, so the gate code is identical for binding and network callers; forward `Cookie` and return `Set-Cookie` explicitly
RPC `WorkerEntrypoint`	BFF → api as typed method calls	Not for the entry chain	`Request` / `Response` are serializable over RPC, so the credential can ride along, but an RPC method is a second entry surface that bypasses the HTTP gate unless the gate is re-run inside each method. Keep RPC for internal non-entry calls (e.g. the CLI-provider path) if at all
Cron Triggers	Expired-session and token sweep	Yes	Free

Enterprise-only or region-limited

- **Access**: self-hosted apps and service tokens are on all plans; 50 users free, then per-seat. Not Enterprise-only.
- **WAF rate limiting** advanced features (counting by JWT claim, response-code conditions, complex expressions) are Business/Enterprise; basic IP-based rules are on all plans.
- **Turnstile Enterprise** adds analytics/SLA only.

- **D1** primary region is chosen at creation (`--location` hint); not movable afterwards without export/import. Read replicas are only in regions Cloudflare chooses.
- **Durable Objects** jurisdiction restrictions (`eu`, `fedramp`) exist on all plans; `fedramp` is Enterprise.
- **Email Sending** quota in beta is per account and not published; the 2026-04 changelog does not list limits.
- Nothing in this design requires Enterprise.

Flagged as unverified

- Current Secrets Store GA status (last confirmed beta 2026-04-24, via a third-party ADR citing the docs).
- Exact Email Sending quota and whether the Free plan can send to onboarded domains.
- Rate Limiting binding behaviour under Smart Placement (whether the "location" becomes the placement location).

5. The threats this design must answer

Every mitigation below has one home in the chain. Where two layers could do it, the table names the authority and marks the other as defence in depth, so no second decider creeps in.

Threat	Mitigation	Lives in	Notes
CSRF against the cookie-authenticated api	`SameSite=Lax` on the session cookie **plus** an origin check: for any state-changing method, the api gate requires `Origin` (or `Sec-Fetch-Site`) to match an allow-list; refuse otherwise. No token dance needed when cookie + origin check are both present	**Gate** (origin check), platform (SameSite)	`Lax` still sends the cookie on top-level GET navigations, so no GET may mutate. SvelteKit's own form-action CSRF check protects the BFF's actions, not api calls the BFF proxies; the api must check for itself, because a CLI or direct browser call never passes the BFF. Over the service binding there is no `Origin`; the BFF must forward the browser's `Origin` header verbatim, and the gate treats a missing header on a binding call as a refusal, not a pass
Session fixation	Issue a new session id on every privilege change: sign-in, passkey registration, password change, role change. Never accept a client-supplied id	**Operation** (`signIn` returns a fresh session; the old one, if any, is deleted)	Because the id is opaque and server-generated, the attacker cannot plant one; the remaining risk is sign-in upgrading an existing anonymous session, which this design does not have

Threat	Mitigation	Lives in	Notes
Cookie scoping across the two Workers	Same-host route split → `__Host-sid; Path=/; Secure; HttpOnly; SameSite=Lax`. The api is the only writer; the BFF forwards	**Gate / entry point** (api sets), BFF (forwards)	With sibling subdomains: `Domain=example.com`, no `__Host-`, and any other subdomain can read and overwrite it (cookie tossing). That is the reason to prefer same-host
Open redirect in OAuth flows	`state` is random, stored server-side (D1 row or signed, short-lived cookie) and compared on callback; the post-login `returnTo` is validated against a relative-path allow-list and never taken from the provider	**Operation** (`oauthCallback`)	Also PKCE on every provider (arctic does this); exact-match redirect URIs in the Google console per environment
Enumeration on sign-up and sign-in	Identical response shape and timing for "no such user", "wrong password", "disabled". Sign-up with an existing email returns the same "check your mail" as a new one and mails "you already have an account". Password-reset likewise	**Operation** (the `Result` carries one generic error code; the entry point maps it to one envelope)	Timing: always run the hash even when the user does not exist (hash a fixed dummy). Rate limiting must key on the *target* email too, or the throttle itself becomes an oracle
Credential stuffing	Three layers: WAF rule on `/api/auth/*` by IP (outer); Rate Limiting binding keyed `ip` and `email` before any store read or hash (gate); per-account failure counter with backoff in D1 or a DO (operation)	**Platform → gate → operation**	Turnstile on the form raises the cost per attempt; passkeys remove the attack entirely. Lockout must be soft (delay, Turnstile escalation), never hard, or an attacker locks victims out

Threat	Mitigation	Lives in	Notes
Verification / magic-link token design	256 bits from `crypto.getRandomValues`, base64url; store SHA-256 of it with `purpose`, `user_id`, `expires_at` (10–15 min), `used_at`; consume in one statement `UPDATE … SET used_at = now WHERE hash = ? AND used_at IS NULL AND expires_at > now RETURNING …`	**Store** (the single-statement consume is what makes it single-use), **operation** (purpose check)	Never put the token in a GET that mutates; the link lands on a page whose POST consumes it, or the GET only renders a confirm button. Invalidate all outstanding tokens of the same purpose when a new one is issued
Sign-out semantics	This device: delete the session row whose id matches the cookie, clear the cookie. All devices: `DELETE FROM sessions WHERE user_id = ?`, and bump nothing else. Disable: `users.disabled = 1` *and* delete the sessions (the gate's join also refuses disabled users, so a missed delete still fails closed)	**Operation** (`signOut`, `signOutAll`, `disableUser`), **store**	Show the user a device list from `sessions` (`user_agent`, `last_seen_at`) so "sign out that one" exists
Session theft / replay	Short sliding window (e.g. 30 days absolute, 7 days idle); id hashed at rest; rotate on privilege change; optionally bind to a coarse `cf-ipcountry` and refuse on change	**Gate** (expiry check), **store** (hash)	Binding to IP breaks mobile users; country is a tolerable compromise

Threat	Mitigation	Lives in	Notes
Passkey challenge replay	Challenge stored server-side, single-use, 5-min expiry; `origin` and `rpId` verified by `@simplewebauthn/server`; signature counter stored and compared	**Operation**, **store**	`rpId` must be stable across BFF and api hosts: use the registrable domain
Access header spoofing (admin)	Verify `Cf-Access-Jwt-Assertion` signature, `aud`, `exp`; refuse if the Worker is reachable without Access (keep the admin routes refusing unless the JWT is present)	**Gate** (JWT verify) then **operation** (role)	Access itself prevents reaching the route; the JWT check prevents trusting a stale configuration
Token leakage in logs	Never log `Cookie`, `Authorization`, or query strings on auth routes; Workers Logs captures what you `console.log`	**Entry point**	Hono's logger middleware logs the path, not headers; keep it that way

The pattern across the table: the **gate** answers "is this a live session from an allowed origin" and nothing else; **operations** answer every question that depends on the user, the purpose or the credential; **stores** make single-use and ownership true by the shape of the statement; the **platform** raises the cost of abuse before your CPU is spent.

6. Where the provisional decisions are wrong

Verdict first: **three of the four decisions stand, one is wrong as stated.** The two-area split (`users` / `auth`) is right. Server-side sessions in a table are right. Ownership by scoped query is right for data and **wrong as the mechanism for authentication**. The decision that is wrong is not in the list: "authentication is a gate that runs once per entry before routing" is right about *when* but under-specified about *what*, and the gap is where the next defect will come from.

Server-side sessions, opaque id, HttpOnly cookie

Strongest case against. A stateful session puts a D1 round trip on the critical path of every request, in a system whose performance rule is counted in round trips. A signed cookie costs zero reads and survives a D1 outage; a 5-minute access token with a refresh token gets you revocation within 5 minutes at the same zero cost. Lucia itself moved to recommend stateless where revocation is not needed, and most of the edge world (Auth.js default, Clerk, Supabase's access tokens) is JWT-first for exactly this reason. You are paying one read per request for a property (instant revocation) that you may use once a year.

Verdict: do not overturn. The owner's rules settle it. "Disable" is a business decision with one authority; a stateless token makes that decision eventually consistent with a horizon the operation cannot control, which is a second, silent authority (the clock). The one read is the same read the first operation would have made to load the actor anyway; the budget test counts it once. What the critique does justify: an **absolute** lifetime, a **sliding** idle window, and the discipline that the gate issues exactly one statement. If the read ever becomes the bottleneck at the large scale, the escape hatch is a KV cache *in front of* D1 that sign-out also invalidates, with the staleness window written into the spec as a number.

`users` and `auth` as two areas

Strongest case against. It is a seam that does not yet carry weight. One user has credentials; splitting them into two areas with two spec files and two store modules forces every sign-in to cross an area boundary (verify credential in `auth`, load person and role from `users`, write session in `auth`) and invites the question "which area owns `disabled`". Better Auth, Auth.js and every hosted provider keep user and account in one module for this reason. A premature seam becomes a place where the same decision is made twice: `auth` refuses a disabled credential and `users` refuses a disabled person.

Verdict: do not overturn, but fix the boundary now. The split is what makes option (d) a deployment change later and what keeps identity-method churn (adding passkeys) out of the `users` spec. The fix: **`disabled` belongs to `users` only, and `auth` never reads it.** The gate joins `sessions` to `users` in one statement and refuses on `users.disabled`; the `auth` operations take a `userId` they were handed and never decide whether that user may act. Write that sentence into the spec table as a rule with a check (a grep-level test that no file under `auth/` references `disabled`).

Ownership enforced by the scoped store query; wrong owner reads as not found

Strongest case against. This is the right rule for resources and the wrong rule for identity. Three failures. First, it only works when there *is* an owner column; the session lookup in the gate has no actor yet, so the gate's query cannot be scoped by actor, it is scoped by the session id, which is the credential. The rule does not cover the one query that matters most. Second, "not found" collapses two facts (does not exist, not yours) that the audit log, the rate limiter and the enumeration defence need to tell apart internally even when the client must not. Third, it pushes a security decision into SQL text, where the spec-and-check rule has the weakest grip: a missing `AND user_id = ?` is a defect that no type and no test catches unless you write a test per query. Capability-style systems and row-level security (Postgres RLS, which D1 does not have) exist because scoping-by-hand is where authorization bugs live.

Verdict: overturn for authentication, keep for data, and add the check. For the gate, the authority is the single joined statement in §2 and the fact it returns; "not found" there means "refuse", not "empty list". For data operations, keep the rule, and make it checkable: every store function that takes an actor has its scope predicate asserted by a test that runs the query as a *different* actor and expects nothing back. That test is the "check" the spec-level rule demands and it is cheap to generate from the spec table. Without it the rule is a convention, which the owner's own design principles say is not enough.

Authentication is a gate, not an operation (decided 2026-11-05)

Strongest case against. Sign-in, sign-out, passkey registration, token consumption are all business decisions with inputs, atomicity requirements and `Result`s; by the owner's own definition they are operations. Calling the whole of authentication "a gate" risks a package that both verifies sessions *and* creates them, which is the gate deciding. It also leaves the CLI-provider case undefined: in-process there is no entry, so there is no gate, so there is no actor.

Verdict: keep the gate, narrow it. The gate is one function: credential in, `Result<Actor, Refusal>` out, one store read, no writes, no side effects, no knowledge of methods. Everything that *produces* a credential (sign-in by any method, verification, rotation, sign-out) is an operation in the `auth` area, declared in the spec table like any other, with its own authorization row (`signOutAll` requires `actor.id = target`). The provider CLI then has a trivial answer: it constructs an `Actor` from the local operator (or runs the same gate against a stored token) and calls operations directly; the gate package exposes the `Actor` type and the `verify` function and nothing else.

One more the owner did not list

The BFF calls the api "today over a service binding." If the api is also public for the CLI, there are two network paths into the same gate with different ambient trust (a binding call has no `Origin`, no `cf-connecting-ip` of the end user, no Turnstile token). The gate must not treat binding origin as authentication, and the BFF must forward `Origin`, the client IP (`CF-Connecting-IP` as a custom header the api accepts *only* over the binding) and the Turnstile token. Otherwise the rate limiter keys every BFF-proxied request to one IP and the origin check has nothing to check. This is a spec row, not a code detail.

Sources

All consulted 2026-11-05. "Doc date" is the page's own last-updated or publish date where shown. Versions were read from the npm registry on 2026-11-05 (`npm view <pkg> version time.modified`).

Source	Type	Doc date	Used for
Workers KV overview and How KV works	Cloudflare docs	2026-08-24	Eventual consistency, 60 s propagation, cacheTtl, negative caching
Redesigning Workers KV	Cloudflare blog	2025-08	Hybrid storage, <5 ms p99, read-your-own-write per location
D1 read replication and changelog 2025-04-10	Cloudflare docs	2026-08-27 / 2025-04-10	Sessions API, bookmarks, beta status
Sequential consistency without borders	Cloudflare blog	2025-04	Replica lag and consistency model
Durable Objects changelog and release notes	Cloudflare docs	through 2026-01-07	SQLite GA 2025-04-07, Free plan, billing 2026-01-07, getByName, Data Studio, best-practices guide
Workers pricing	Cloudflare docs	2026-10-02	All per-million prices and included quotas for Workers, D1, KV, DO, Queues, Workflows

Source	Type	Doc date	Used for
Workers limits	Cloudflare docs	2026	10 ms Free / 30 s default / 5 min max CPU, 128 MB memory, waitUntil 30 s
Rate Limiting binding and GA changelog 2025-09-19	Cloudflare docs	2025-09-19	GA, `ratelimits` config, per-location semantics, period 10/60
Workers RPC	Cloudflare docs	2026	Serializable types incl. Request/Response, WorkerEntrypoint
Cloudflare Access for Workers and Application paths	Cloudflare docs	2026	Protect one Worker / all Workers / a hostname or path; wildcard rules; no ports or query strings
Email Sending public beta changelog 2026-04-16 and local development	Cloudflare docs	2026-04-16	Beta status, binding, `remote: true`, Paid-plan requirement for arbitrary recipients
Announcing Cloudflare Email Service private beta	Cloudflare blog	2025-09-25	Private beta date
Workers Secrets and Secrets Store llms-full	Cloudflare docs	2026-07-03	Per-Worker secrets vs Secrets Store binding; deploy permission
Queues on Free plan changelog 2026-02-04 and Workflows billing updates 2026-07-07	Cloudflare changelog	as dated	Status changes
Turnstile plans	Cloudflare docs	2026	Free on all plans
SvelteKit 3 on Netlify	Vendor changelog (third party)	2026-10-02	SvelteKit 3 release date and breaking changes; the svelte.dev announcement itself was not fetched

Source	Type	Doc date	Used for
Better Auth 1.6 blog and GitHub releases 1.6.23–1.7.10	Library blog / releases	2026	Release tracks, Cloudflare fixes, D1 adapter fixes
Vercel acquires Better Auth and Better Auth is joining Vercel	Vendor blogs	2026-07-07	Ownership, MIT continues
Auth.js integrations and Auth.js security update July 2026	Library docs / blog	2026-07-21	`@auth/sveltekit` experimental status, maintenance under Better Auth, 0.41.3 advisories
Lucia README, A fresh start #1714, lucia-auth.com	Repo / site	2024-10-20; site updated 2026-07	Deprecation and the learning-resource direction; Oslo and Arctic maintained
OpenAuth docs and anomalyco/openauth	Library docs / repo	beta; last npm publish 2025-03-22	Issuer model, KV storage, Hono base, release cadence
Hashing passwords on Cloudflare Workers (blog, 2024-02-21); nuxt-cryptography README (2026); sentinel v0.7.1 release note (2026-08-15)	Third-party	as dated	Independent reports of the 100,000-iteration PBKDF2 cap on Workers; **no Cloudflare doc found stating it**
Native email and password authentication on Cloudflare Workers	Blog	2026	OWASP 600k figure as cited; pepper pattern; rate-limit-before-hash; used as corroboration only
npm registry	Registry	2026-11-05	All versions in the Summary

Not fetched directly and stated from memory, marked as such in the text: OWASP Password Storage Cheat Sheet parameters (argon2id m=19 MiB t=2 p=1; scrypt N=2^17; bcrypt 10; PBKDF2-SHA256 600k), Queues GA 2024-10, Workflows GA 2025-04, Zero

Trust Free plan 50 users, `hash-wasm` argon2id timings on Workers, D1 primary-region latency ranges.